자율운항선박 기술개발사업

프로토타입 INHA 모듈 사용방법

2021.08.27.(금)

해양시스템제어 연구실
석사과정 김지수

‘inha_modules_ROS’ 사용방법

원격 저장소 목록

➢ SW 구성

- inha_modules_ROS
  - 인하대 충돌회피 모듈 및 전체 loop 구성
  - git clone ssh://kass@165.246.99.174:10022/home/kass/git-repos/git/inha_modules_ROS.git

- kass_msgs
  - 인하대, KAIST, 부경대, KRISO message 파일이 모여져 있는 폴더
  - git clone ssh://kass@165.246.99.174:10022/home/kass/git-repos/git/kass_msgs.git

‘inha_modules_ROS’ 사용방법

실행 방법

- Step 1) `main_parameter.yaml` 수정
  - update_rate : ros update 주기 (필요시 수정)
  - CA_starts : 임의의 충돌회피 시점[m]로 반영 (필요시 수정)
  - Ship_info : 시뮬레이션 하고 싶은 선박 정의
  - waypoint_List : 각 선박들의 추종해야하는 전역 경로의 waypoint 설정

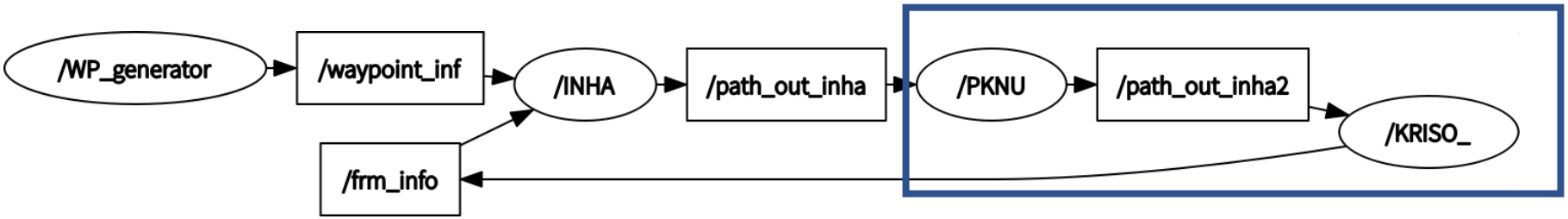
- Step 2) `roslaunch inha_modules inha_modules.launch ` 실행

- Step 3) ‘roslaunch plotjuggler_ros plotjuggler.launch` 실행
  - 설치방법은 아래 링크 참조
    - https://github.com/PlotJuggler/plotjuggler-ros-plugins
    - `Debians`버전으로 설치하지 말 것!(unstable)

‘inha_modules_ROS’ 사용방법

시뮬레이션 환경 구성 (KRISO 노드)

➢ n척의 선박을 운용할 수 있는 시뮬레이션 환경 구성 (운항 모델 반영)



- 인하대
  - 자선/타선에 대한 desired value 생성 중

- 부경대
  - 자선/타선의 desired value를 바탕으로 제어 명령 생성

- KRISO
  - 자선/타선의 운항모델만 반영되어 있음

- **시뮬레이션 환경**
  - 자선/타선의 운항모델 반영
  - 자선/타선의 desired value를 바탕으로 제어 명령 생성

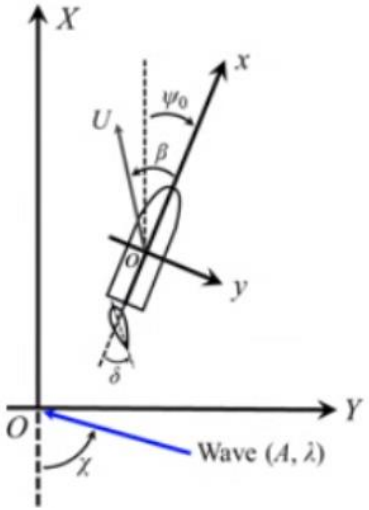
‘inha_modules_ROS’ 사용방법

시뮬레이션 환경 구성 (KRISO 노드)

➢ n척의 선박을 운용할 수 있는 시뮬레이션 환경 구성 (운항 모델 반영)

- 1) 초기정보 생성
- 2) t 초에서 `desired_psi`, `desired_speed`값으로 t+1의 position 계산
  - Speed, Steering (Kp=1), Heading제어기를 포함하고 있음

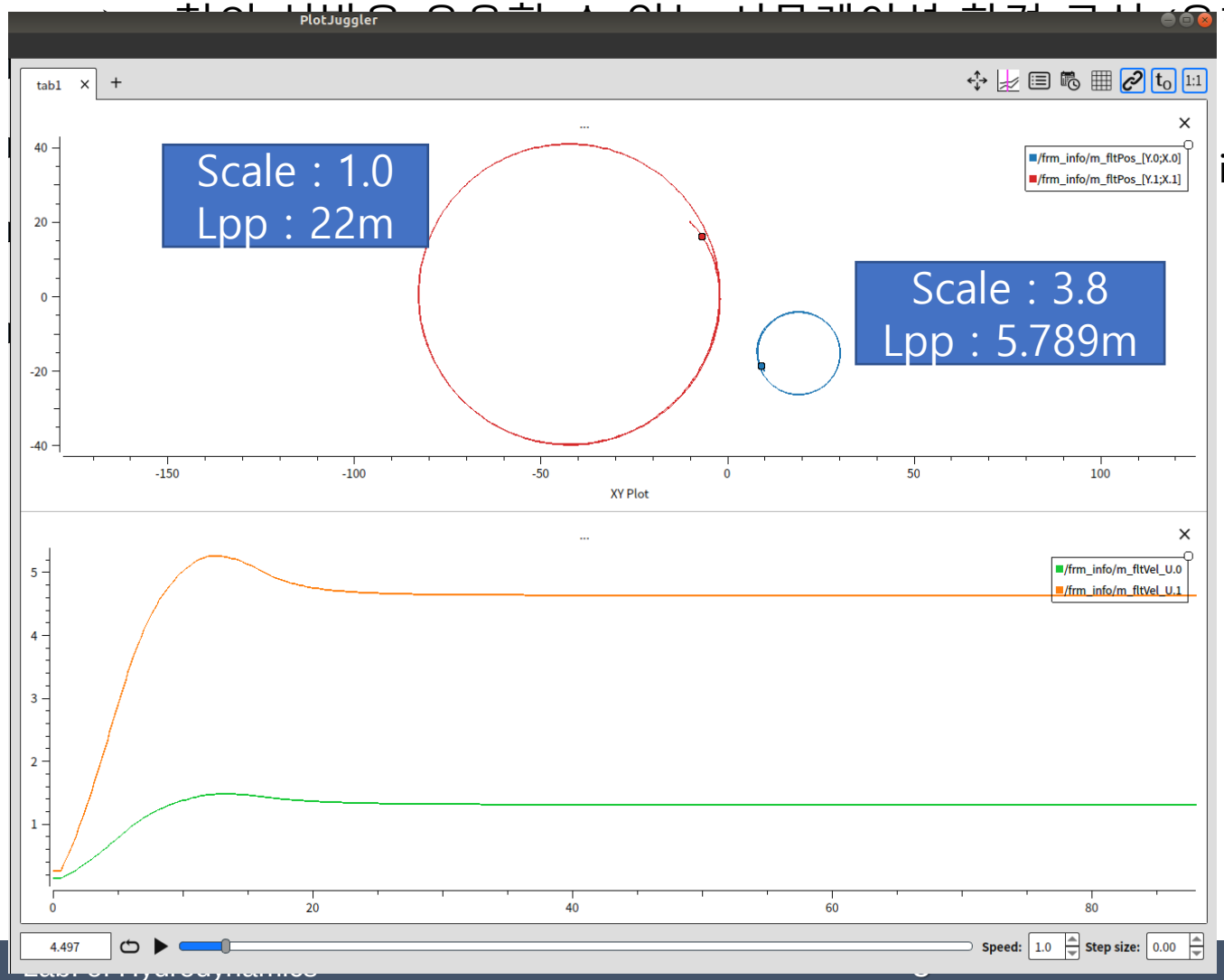
```
OS_info : {
  ship_L : 5.7895, ## 모형선의 길이 //향후 AIS data로 대체
  ship_scale : 3.8,
  ship_ID : 2000, ## 자선의 ship ID 정의
  ship_domain_radius : 1.0,  ## 이 부분은 향후 ship domain으로 포함이 되어야 함
  target_speed : 6.0,  #[knots]
  initial_start_x : -20.0,    # [m]
  initial_start_y : 10.0,    # [m]
  initial_start_U : 0.5 ,    # [knots] // 너무 작으면 발산함
  initial_start_psi : -28.0   # [deg]
}

TS1_info : {
  ship_L : 22.0, ## 모형선의 길이 //향후 AIS data로 대체
  ship_scale : 1.0,
  ship_ID : 2001, ## 자선의 ship ID 정의
  ship_domain_radius : 1.0,  ## 이 부분은 향후 ship domain으로 포함이 되어야 함
  target_speed : 6.0,       #[knots]
  initial_start_x : 20.0,    # [m]
  initial_start_y : -10.0,    # [m]
  initial_start_U : 0.5,    # [knots]  // 너무 작으면 발산함
  initial_start_psi : 150.0   # [deg]
}
```

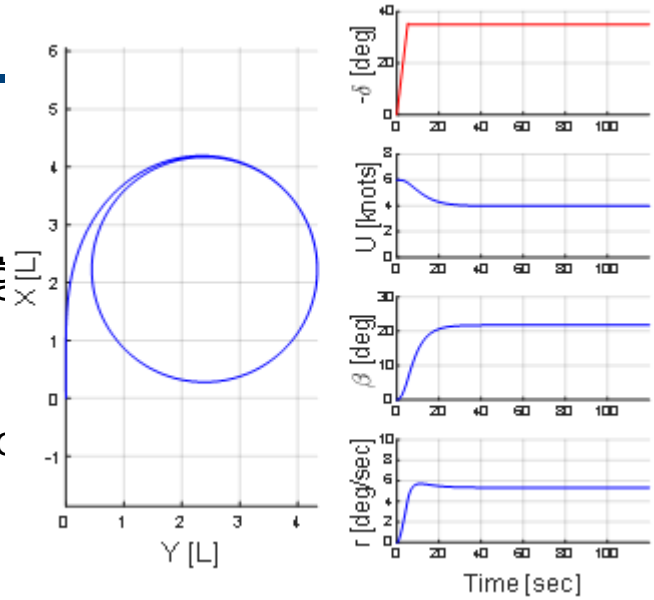


<NED 좌표계>

‘inha_modules_ROS’ 사용방법

시뮬레이션 환경 구성 (KRISO 노드)





선회 (6노트)	35° 우현	35° 좌현
전진거리 [L]	4.07	4.07
전술직경 [L]	4.21	4.21
속도감소비 (U_f/U_0) [-]	0.66	0.66
최종편류각 [°]	21.76	21.76
최종선회율 ($r/(U_f/L)$) [-]	0.51	0.51

‘inha_modules_ROS’ 사용방법

n척의 선박을 운용할 수 있는 시뮬레이션 환경 구성 방법

➢ 1. `main_parameter.yaml` 파일 수정

- `Ship_info` 생성
  - Ship_L & ship_scale 대응하게 맞춰주고, target speed와 initial condition 정의
- `waypoint_List` 추가
  - 추가된 선박에 대한 ‘wpts_x_ts’ & ‘wpts_y_ts’ 정의 (전역경로 추종 waypoint 생성)

```
OS_info : {
  ship_L : 5.7895, ## 모형선의 길이 //향후 AIS data로 대체
  ship_scale : 3.8,
  ship_ID : 2000, ## 자선의 ship ID 정의
  ship_domain_radius : 1.0,  ## 이 부분은 향후 ship domain으로 포함이 되어야 함
  target_speed : 6.0,  #[knots]
  initial_start_x : -20.0,    # [m]
  initial_start_y : 10.0,    # [m]
  initial_start_U : 0.5 ,    # [knots] // 너무 작으면 발산함
  initial_start_psi : -28.0   # [deg]
}
```

```
waypoint_List: {
  wpts_x_os: [
    -6.14, 20.44, 27.0
  ],
  wpts_y_os: [
    2.63,  -10.46, -13.5
  ],
```

‘inha_modules_ROS’ 사용방법

n척의 선박을 운용할 수 있는 시뮬레이션 환경 구성 방법

➢ 2. ‘WP_pub.py’ 파일 수정

- 전역경로 waypoint list를 publish해주는 노드임
  - Red box를 main_parameter에 맞게 수정

```python
def main():
    update_rate = rospy.get_param("update_rate")
    pub = rospy.Publisher('/waypoint_info', group_wpts_info, queue_size=10)

    os = wpts_info()
    ts = wpts_info()
    ts2 = wpts_info()

    rospy.init_node('WP_generator', anonymous=False)
    rate = rospy.Rate(update_rate) # 2초에 한번씩 데이터 줘라

    while not rospy.is_shutdown():
        topic = group_wpts_info()
        topic.header.stamp = rospy.Time.now()  # http://wiki.ros.org/rospy/Overview/Time
        topic.header.frame_id = "group_waypoint_list"

        wpts_x_os = rospy.get_param("waypoint_List/wpts_x_os")
        wpts_y_os = rospy.get_param("waypoint_List/wpts_y_os")
        wpts_x_ts = rospy.get_param("waypoint_List/wpts_x_ts")
        wpts_y_ts = rospy.get_param("waypoint_List/wpts_y_ts")
        wpts_x_ts2 = rospy.get_param("waypoint_List/wpts_x_ts2")
        wpts_y_ts2 = rospy.get_param("waypoint_List/wpts_y_ts2")
```

```python
os.shipID = rospy.get_param("OS_info/ship_ID")
os.wpts_x = wpts_x_os
os.wpts_y = wpts_y_os

ts.shipID = rospy.get_param("TS1_info/ship_ID")
ts.wpts_x = wpts_x_ts
ts.wpts_y = wpts_y_ts

ts2.shipID = 2002
ts2.wpts_x = wpts_x_ts2
ts2.wpts_y = wpts_y_ts2

topic.group_wpts_info = [os, ts, ts2]
```

‘inha_modules_ROS’ 사용방법

n척의 선박을 운용할 수 있는 시뮬레이션 환경 구성 방법

➢ 3. ‘KRISO.py’ 수정

- 초기 운항 정보 생성 및 Red box를 main_parameter에 맞게 수정

```
## 자선 초기 운항정보 생성
OS_ID = rospy.get_param("OS_info/ship_ID")
OS_scale = rospy.get_param("OS_info/ship_scale")
OS_X = rospy.get_param("OS_info/initial_start_x")
OS_Y = rospy.get_param("OS_info/initial_start_y")
OS_U = rospy.get_param("OS_info/initial_start_U") * 0.5144 / sqrt(OS_scale)
OS_psi = rospy.get_param("OS_info/initial_start_psi")

## 타선1 초기 운항정보 생성
TS1_ID = rospy.get_param("TS1_info/ship_ID")
TS1_scale = rospy.get_param("TS1_info/ship_scale")
TS1_X = rospy.get_param("TS1_info/initial_start_x")
TS1_Y = rospy.get_param("TS1_info/initial_start_y")
TS1_U = rospy.get_param("TS1_info/initial_start_U") * 0.5144 / sqrt(TS1_scale)
TS1_psi = rospy.get_param("TS1_info/initial_start_psi")
```

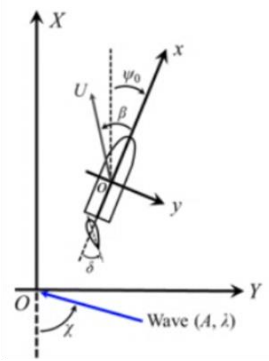
```
# 자선에 대한 MMG기반 t+1 운항정보 생성
OS_desired_Heading = kriso.path_out_inha_dic[f'{OS_ID}'].desired_heading
OS_desired_spd = kriso.path_out_inha_dic[f'{OS_ID}'].desired_spd
OS_X, OS_Y, OS_U, OS_psi = simulation_os.moving_ships(OS_desired_Heading, OS_desired_spd)

# 타선1에 대한 MMG기반 t+1 운항정보 생성
TS_desired_Heading = kriso.path_out_inha_dic[f'{TS1_ID}'].desired_heading
TS_desired_spd = kriso.path_out_inha_dic[f'{TS1_ID}'].desired_spd
TS1_X, TS1_Y, TS1_U, TS1_psi = simulation_ts.moving_ships(TS_desired_Heading, TS_desired_spd)
```

```
### 시뮬레이션 수행을 위한 각 class들의 인스턴스화
simulation_os = ShipSimulation(OS_X, OS_Y, OS_U, OS_psi, initial_delta, OS_scale, dt)
simulation_ts = ShipSimulation(TS1_X, TS1_Y, TS1_U, TS1_psi, initial_delta, TS1_scale, dt)
# simulation_ts2 = ShipSimulation(TS2_X, TS2_Y, TS2_U, TS2_psi, initial_delta, TS2_scale, dt)
```

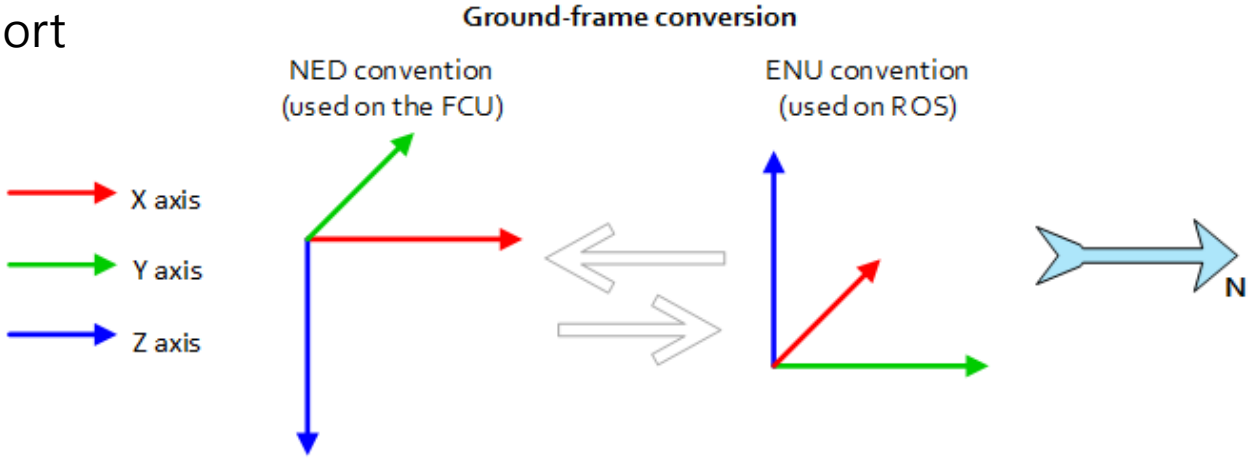
‘inha_modules_ROS’ 사용방법

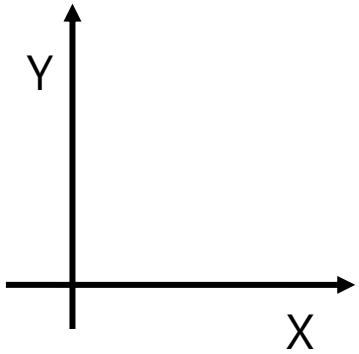
좌표계 변환 문제

➢ Issue report



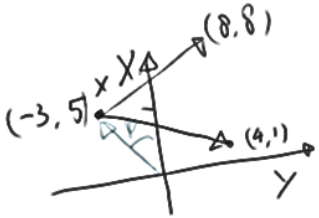
<운항모델 좌표계>





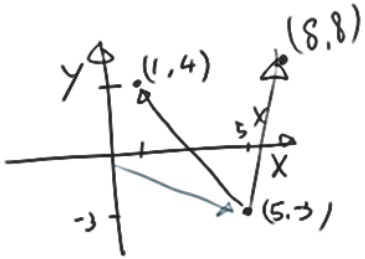
<VO 좌표계>

- KRSIO 및 기타 좌표계 (NED 좌표계)
  - Ex) CRI 계산 등



$V_{des} = [7, -4]$

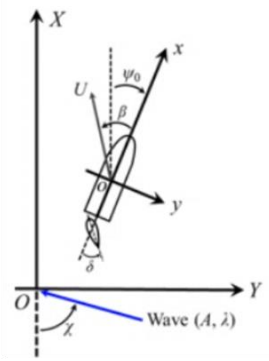
- VO 좌표계 (일반적인 XY좌표계)



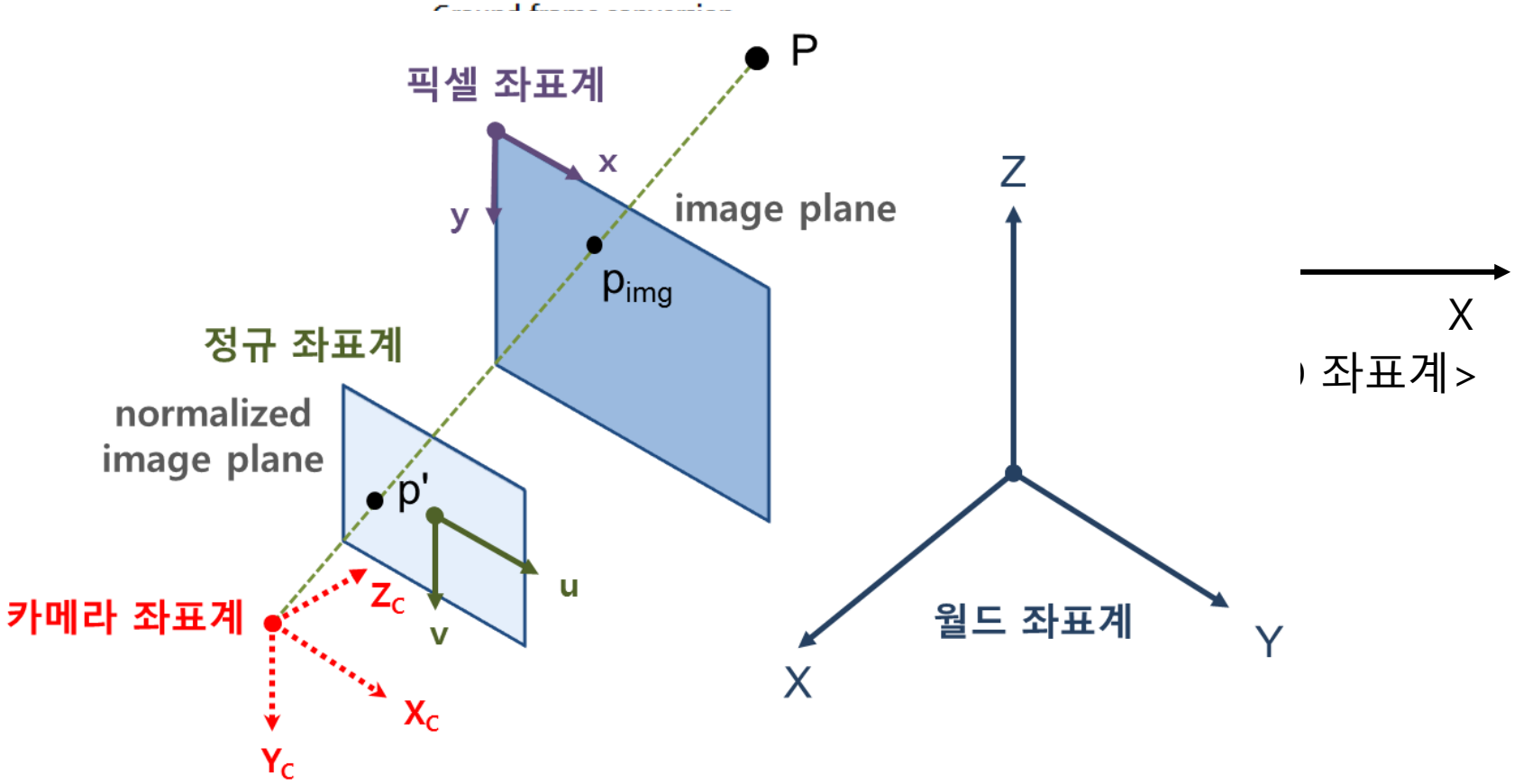
$V_{des} = (-4, 7)$

좌표계 변환 문제

➢ Issue report



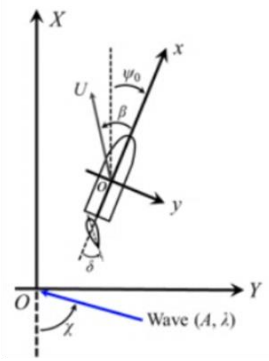
<운항모델 좌표계>



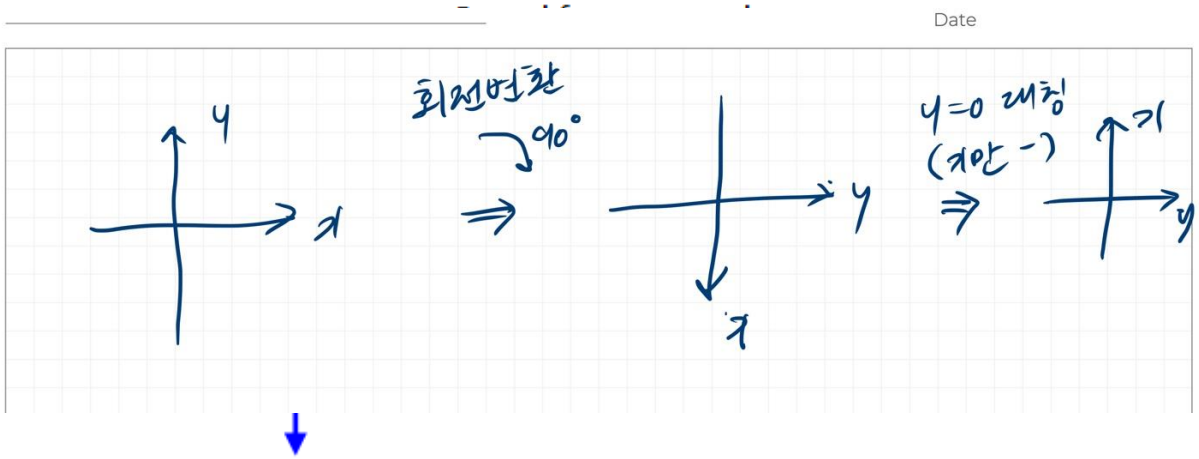
‘inha_modules_ROS’ 사용방법

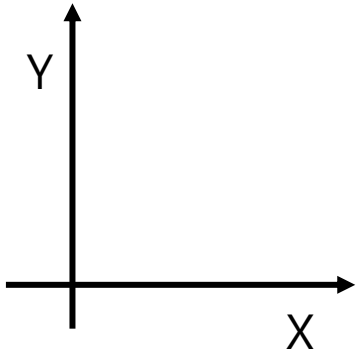
좌표계 변환 문제

➢ Issue report



<운항모델 좌표계>





<VO 좌표계>

- 단, `INHA.py`에 모두 좌표계를 변환하지 않음 (input값 유지)
  - 알고리즘에 따라 NED좌표계가 필요할 수 있으므로.

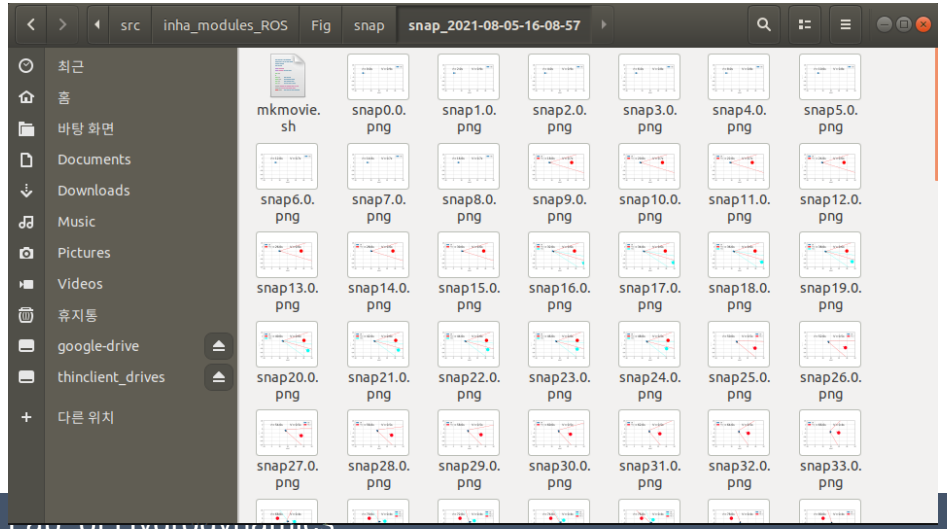
- `VO.py`의 input & output만 수정하였음

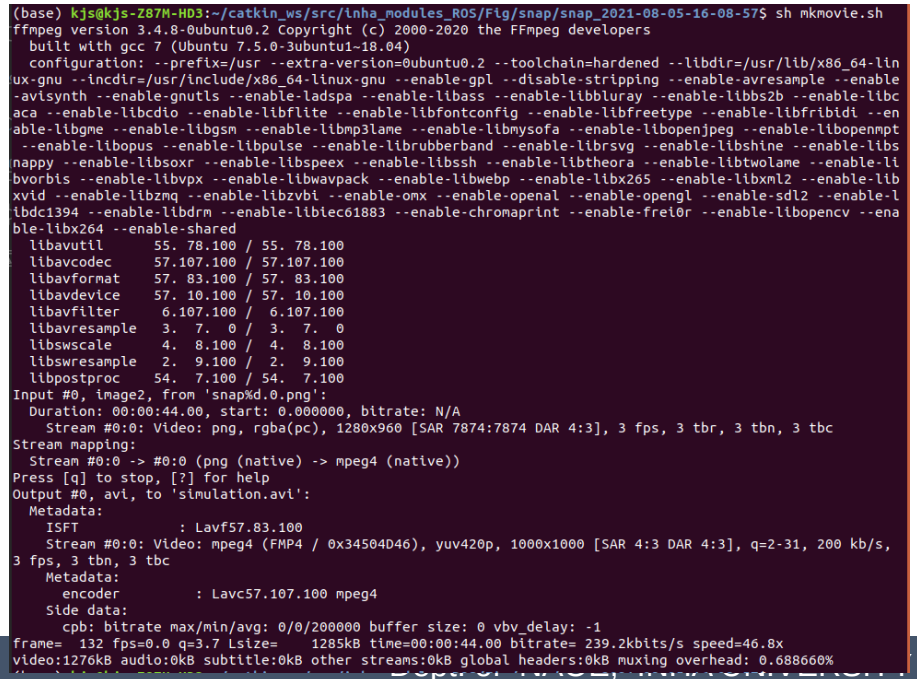
‘inha_modules_ROS’ 사용방법

`mkmovie.sh` 사용법

➢ 이미지를 frame 단위로 저장

- `history_collector.visualize_traj_dynamic()` 이미지를 frame 단위로 저장함
  - t step에서 1) 자/타선의 위치, 2) 방향벡터, 3) Collision Cone, 4) 정적장애물 등을 plot
  - 따라서, 총 시뮬레이션 time 동안 dt만큼 이미지를 frame 단위로 생성하여 저장함
- 동영상으로 만드는 단순한 실행 함수 Terminal에서 `sh mkmovie.sh`를 실행하면 됨
  - 안에 옵션은 ‘ffmpeg’ 옵션을 참고할 것
  - https://peche326.tistory.com/58





```
(base) kjs@kjs-Z87M-HD3:~/catkin_ws/src/inha_modules_ROS/Fig/snap/snap_2021-08-05-16-08-57$ sh mkmovie.sh
ffmpeg version 3.4.8-0ubuntu0.2 Copyright (c) 2000-2020 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1~18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.2 --toolchain=hardened --libdir=/usr/lib/x86_64-lin
ux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresample --enable
-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libc
aca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --enable-libfribidi --en
able-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt
 --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg --enable-libshine --enable-libs
nappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-li
bvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --enable-libx265 --enable-libxml2 --enable-lib
xvid --enable-libzmq --enable-libzvbi --enable-omx --enable-openal --enable-opengl --enable-sdl2 --enable-l
ibdc1394 --enable-libdrm --enable-libiec61883 --enable-chromaprint --enable-frei0r --enable-libopencv --ena
ble-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Input #0, image2, from 'snap%d.0.png':
  Duration: 00:00:44.00, start: 0.000000, bitrate: N/A
    Stream #0:0: Video: png, rgba(pc), 1280x960 [SAR 7874:7874 DAR 4:3], 3 fps, 3 tbr, 3 tbn, 3 tbc
Stream mapping:
  Stream #0:0 -> #0:0 (png (native) -> mpeg4 (native))
Press [q] to stop, [?] for help
Output #0, avi, to 'simulation.avi':
  Metadata:
    ISFT            : Lavf57.83.100
    Stream #0:0: Video: mpeg4 (FMP4 / 0x34504D46), yuv420p, 1000x1000 [SAR 4:3 DAR 4:3], q=2-31, 200 kb/s,
3 fps, 3 tbn, 3 tbc
    Metadata:
      encoder         : Lavc57.107.100 mpeg4
    Side data:
      cpb: bitrate max/min/avg: 0/0/200000 buffer size: 0 vbv_delay: -1
frame=  132 fps=0.0 q=3.7 Lsize=    1285kB time=00:00:44.00 bitrate= 239.2kbits/s speed=46.8x
video:1276kB audio:0kB subtitle:0kB other streams:0kB global headers:0kB muxing overhead: 0.688660%
```

Thank you

원격 저장소 접속 방법

KASS 원격 저장소 clone 방법

➢ Window PowerShell or Ubuntu Terminal실행

- git clone ssh://kass@165.246.99.174:10022/home/kass/git-repos/git/KASS_CA.git
  - git clone ssh://[user]@[ip 주소]:port/git 파일 저장위치
  - 처음 ssh 접속시 public key 등록할 것인지 물어보는데, 이때 그냥 `yes`하면 됨
    - yes 를 입력 엔터치면 다음부터는 묻지 않으며, no로 하면 매번 묻는다.
  - kass@165.246.99.174 password
    - q1w2e3r4

```
PS C:₩Users₩KJS₩test> git clone ssh://kass@165.246.99.174:10022/home/kass/git-repos/git/KASS_CA.git
Cloning into 'KASS_CA'...
kass@165.246.99.174's password:
remote: Counting objects: 100, done.
remote: Compressing objects: 100% (98/98), done.
remote: Total 100 (delta 33), reused 0 (delta 0)R
Receiving objects: 100% (100/100), 711.20 KiB | 12.26 MiB/s, done.
Resolving deltas: 100% (33/33), done.
```

원격 저장소 접속 방법

원격 저장소에 push/pull 및 branch 생성

- git push origin {branch name}
  - 수정된 commit을 원격 저장소에 업데이트
- git fetch origin
  - 원격 저장소에서 업데이트된 내용 확인
- git pull origin {branch name}
  - 원격 저장소에서 업데이트된 내용 다운 및 반영
- git merge {branch name}
  - Branch 합치기

- 자세한 내용은 아래 링크 참조
  - https://goddaehee.tistory.com/274?category=381481
  - https://uang.tistory.com/11?category=799977